```
def updateCounts(newCountsDs: Dataset[TransactionCount], epoch_id: Long): Unit = {
  val newCountsDf = newCountsDs.toDF
  val aggregationTable = DeltaTable.forName(table_name)
  aggregationTable.alias("t")
    .merge(newCountsDf.alias("m"), "m.CustomerID = t.CustomerID")
    .whenMatched().updateAll()
    .whenNotMatched().insertAll()
    .execute()
}
```

Figure 5.24 – The updateCounts function upserts the new transaction counts into the transaction_count_ft table

4. The last piece of the stream is the write, or, more clearly, the update. The write stream applies the `updateCounts` function. Delta tables do not support streaming updates directly, so we need to use a `foreachBatch` function. The `foreachBatch` function is like a streaming `for` loop applying the function to each micro-batch of data in the stream. This write stream is similar to `flatMapGroupsWithState` without grouping the data or maintaining the state. We are simply updating the CDC result table. Notice the checkpointing is handled for us, and our stream is triggered every 10 seconds, meaning every 10 seconds is a new micro-batch of the data. The query name is optional. It shows up in the SparkUI.

```
flatMapGroupsWithStateResultDf.writeStream
  .foreachBatch(updateCounts _)
  .option("checkpointLocation", f"$outputPath/checkpoint")
  .trigger(Trigger.ProcessingTime("10 seconds"))
  .queryName("flatMapGroups")
  .start()
```

Figure 5.25 – The write stream applies the updateCounts function using foreachBatch

5. In addition to the writing out to the CDC table, we also want a historical record of the transaction values:

```
flatMapGroupsWithStateResultDf.writeStream
  .option("checkpointLocation", f"$outputPath2/checkpoint")
  .trigger(Trigger.ProcessingTime("10 seconds"))
  .queryName("flatMapGroupsHistoryTable")
  .table(history_table_name)
```

Figure 5.26 – The write stream records all of the transactionCounts values to the Delta table